
Tribhuvan University
Faculty of Humanities & Social Sciences
OFFICE OF THE DEAN

OOP in Java

BCA — Semester 3 — 2019 — Answer Sheet

Subject Code: CACS204

Group A

Attempt ALL the questions. [10 × 1 = 10]

1. i) Which one of the following is not a valid java operator?

a) ● (Dot) b) >>> c) instanceof d) system

ii) Which one of the following keyword is not used in exception handling?

a) throws b) throw c) try d) tries

iii) Which one of the following is not a valid jumping statement in Java?

a) continue b) break c) goto d) return

iv) Which one of the following access specifier is appropriate for members of superclass to access only from subclass?

a) private b) protected c) public d) default

v) Which one of the following is not a collection class defined in java?

a) Linked List b) Hash Set c) Tree Set d) Graph Set

vi) Which one of the following inheritance uses interface?

a) single inheritance b) multi-level inheritance c) multiple inheritance d) hierarchical inheritance

vii) Which one of the following method is called only once during the run time of your applet?

a) stop() b) paint() c) init() d) start()

viii) Which of these method of class String is used to compare two String objects for their equality?

a) equals() b) Equals() c) is Equal() d) Is Equal()

ix) What is the default value of priority variable MIN_PRIORITY and MAX_PRIORITY?

a) 0 & 63 b) 1 & 10 c) 0 & 1 d) 1 & 32

x) Which one of the following is not java swing menu control?

a) Menu Bar b) Menu c) Menu Item d) Menu Scroll

[10 × 1 = 10]

Answer: Answers:

i) **d) system** — system is not an operator; it is a class (System) in java.lang package. The valid operators are dot (.), unsigned right shift (>>>), and instanceof.

ii) **d) tries** — The keywords used in exception handling are try, catch, finally, throw, and throws. "tries" is not a Java keyword.

iii) **c) goto** — goto is a reserved keyword in Java but it is not used as a valid jumping statement. Java uses break, continue, and return for control flow.

iv) **b) protected** — The protected access specifier allows members to be accessed within the same package and by subclasses (even in different packages).

v) **d) Graph Set** — LinkedList, HashSet, and TreeSet are all valid collection classes in java.util package. Graph Set does not exist.

vi) **c) multiple inheritance** — Java does not support multiple inheritance of classes, but it achieves multiple inheritance through interfaces.

vii) c) init() — The `init()` method is called once when the applet is first loaded. `start()` and `stop()` can be called multiple times, and `paint()` is called whenever the applet needs to be redrawn.

viii) a) equals() — The `equals()` method (case-sensitive, lowercase) of the `String` class compares the actual content of two `String` objects for equality.

ix) b) 1 & 10 — In Java, `Thread.MIN_PRIORITY` = 1 and `Thread.MAX_PRIORITY` = 10. The default normal priority is 5 (`NORM_PRIORITY`).

x) d) Menu Scroll — `JMenuBar`, `JMenu`, and `JMenuItem` are valid Swing menu components. `Menu Scroll` is not a standard Swing menu control.

Group B

Attempt any SIX questions. [$6 \times 5 = 30$]

2. Define OOP. Explain features of Object Oriented Programming Language. [1 + 4]

[5]

Answer: Definition of OOP:

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around data (objects) rather than functions and logic. An object is an instance of a class that contains both data (attributes) and methods (behaviors).

Features of OOP:

1. Encapsulation: Wrapping data and methods into a single unit (class) and restricting direct access to some components using access modifiers (`private`, `protected`, `public`). This protects data integrity.

2. Inheritance: The mechanism by which one class (subclass/child) acquires the properties and behaviors of another class (superclass/parent). It promotes code reusability. Java supports single, multi-level, hierarchical, and hybrid inheritance.

3. Polymorphism: The ability to take multiple forms. In Java, it is achieved through method overloading (compile-time polymorphism) and method overriding (runtime polymorphism).

4. Abstraction: Hiding implementation details and showing only functionality. Achieved using abstract classes and interfaces.

5. Class and Object: A class is a blueprint/template, while an object is a real-world entity created from that class.

3. Explain different types of looping statements used in java. [5]

[5]

Answer: Looping Statements in Java:

1. for loop: Used when the number of iterations is known beforehand.

```
for(int i=0; i<5; i++) { System.out.println(i); }
```

2. while loop: Used when the number of iterations is not known and depends on a condition. The condition is checked before entering the loop.

```
int i = 0; while(i < 5) { System.out.println(i); i++; }
```

3. do-while loop: Similar to while, but the condition is checked after executing the loop body. It guarantees at least one execution.

```
int i = 0; do { System.out.println(i); i++; } while(i < 5);
```

4. Enhanced for loop (for-each): Used to iterate over arrays or collections without using an index variable.

```
int[] arr = {1, 2, 3, 4, 5}; for(int num : arr) { System.out.println(num); }
```

5. Nested loops: A loop inside another loop. Useful for working with multi-dimensional arrays and patterns.

4. Define Abstract Class. Explain different types of Access controls available in java. [1 + 4]

[5]

Answer: Abstract Class:

An abstract class is a class that cannot be instantiated directly. It is declared using the **abstract** keyword and may contain both abstract methods (without body) and concrete methods (with body). It serves as a base class for subclasses to extend and implement the abstract methods.

```
abstract class Animal { abstract void makeSound(); void sleep() { System.out.println("Sleeping..."); } }
```

Access Controls (Access Modifiers) in Java:

1. private: Accessible only within the same class. Used for encapsulation.

2. default (package-private): Accessible only within the same package. No keyword is used.

3. protected: Accessible within the same package and by subclasses in different packages.

4. public: Accessible from anywhere — any class, any package.

Modifier	Class	Package	Subclass	World	public	Yes	Yes	Yes	Yes	protected	Yes	Yes	Yes	No	default	Yes	Yes	No	No
	private	Yes	No	No	No														

5. How polymorphism is achieved in Java? Explain with example. [1 + 4]

[5]

Answer: Polymorphism in Java:

Polymorphism means "many forms." It allows objects of different classes to be treated as objects of a common superclass. In Java, polymorphism is achieved in two ways:

1. Compile-time Polymorphism (Method Overloading):

Multiple methods in the same class have the same name but different parameter lists (different number, type, or order of parameters).

```
class Calculator { int add(int a, int b) { return a + b; } int add(int a, int b, int c) { return a + b + c; } double add(double a, double b) { return a + b; } } Calculator c = new Calculator(); c.add(2, 3); // calls first method c.add(2, 3, 4); // calls second method c.add(2.5, 3.5); // calls third method
```

2. Runtime Polymorphism (Method Overriding):

A subclass provides a specific implementation of a method that is already defined in its superclass. The JVM determines which method to call at runtime based on the actual object's class.

```
class Animal { void makeSound() { System.out.println("Animal sound"); } } class Dog extends Animal { void makeSound() { System.out.println("Bark"); } } Animal a = new Dog(); a.makeSound(); // Output: Bark (runtime binding)
```

6. Define Exception Handling. Write a program to illustrate the use of exception handling. [1 + 4]

[5]

Answer: Exception Handling:

Exception handling is a mechanism in Java to handle runtime errors (exceptions) gracefully without crashing the program. It uses try, catch, finally, throw, and throws keywords to manage exceptional conditions.

Program illustrating Exception Handling:

```
import java.util.Scanner; public class ExceptionDemo { public static void main(String[] args) { Scanner sc = new Scanner(System.in); try { System.out.print("Enter first number: "); int a = sc.nextInt(); System.out.print("Enter second number: "); int b = sc.nextInt(); int result = a / b; System.out.println("Result: " + result); } catch (ArithmeticException e) { System.out.println("Error: Division by zero is not allowed!"); } catch (Exception e) { System.out.println("Error: " + e.getMessage()); } finally { System.out.println("Execution completed."); sc.close(); } } }
```

Explanation:

- **try:** Contains code that might throw an exception.
- **catch:** Handles specific exceptions (ArithmeticException for division by zero).
- **finally:** Always executes, used for cleanup (closing resources).
- **throw:** Used to explicitly throw an exception.
- **throws:** Declares exceptions that a method might throw.

7. Define the use of static keyword. Write any four String methods used in java with example. [1 + 4]

[5]

Answer: static Keyword in Java:

The **static** keyword is used to create members (variables and methods) that belong to the class itself rather than to any specific instance (object). Static members are shared across all instances of the class and can be accessed without creating an object.

Uses of static:

- **static variable:** A single copy shared by all objects (class variable).
- **static method:** Can be called without creating an object. Can access only static members directly.
- **static block:** Executed once when the class is loaded, used for initialization.
- **static class:** Only nested classes can be static.

```
class Student { static int count = 0; // static variable Student() { count++; } static void showCount() { // static method System.out.println("Total students: " + count); } }
```

Four String Methods with Examples:

1. length(): Returns the number of characters in the string.

```
String s = "Hello"; int len = s.length(); // len = 5
```

2. charAt(int index): Returns the character at the specified index.

```
String s = "Java"; char c = s.charAt(0); // c = 'J'
```

3. substring(int beginIndex, int endIndex): Returns a portion of the string.

```
String s = "Programming"; String sub = s.substring(0, 4); // sub = "Prog"
```

4. equals(String str): Compares two strings for content equality.

```
String s1 = "Java"; String s2 = "Java"; boolean result = s1.equals(s2); // result = true
```

8. Define super, final and this keyword in java. Explain the concept of MVC in brief. [1+1+1+2]

[5]

Answer: super Keyword:

The **super** keyword is used to refer to the immediate parent class object. It is used to:

- Call the parent class constructor: **super();**
- Access parent class methods: **super.methodName();**
- Access parent class variables: **super.variableName;**

final Keyword:

The **final** keyword is used to restrict modification:

- **final variable:** Cannot be changed after initialization (constant).
- **final method:** Cannot be overridden by subclasses.
- **final class:** Cannot be extended (inherited).

this Keyword:

The **this** keyword refers to the current object instance. It is used to:

- Differentiate instance variables from local variables with the same name.
- Call another constructor in the same class: **this();**
- Pass the current object as a parameter.

MVC (Model-View-Controller) Architecture:

MVC is a software design pattern that separates an application into three interconnected components:

1. **Model:** Represents the data and business logic. It manages the application's data, rules, and logic. When data changes, it notifies the View.
2. **View:** Represents the user interface (UI). It displays data from the Model to the user and sends user commands to the Controller.
3. **Controller:** Acts as an intermediary between Model and View. It receives user input from the View, processes it (using Model), and updates the View accordingly.

Benefits: Separation of concerns, easier maintenance, parallel development, and code reusability.

Group C

Attempt any TWO questions. [2 × 10 = 20]

9. a) Define multithreading. Write a java program to show the inter-thread communication. [1+4]
b) Define Stream. Write a program in java to copy the content from one file to another. [1+4]

[10]

Answer: a) Multithreading:

Multithreading is a Java feature that allows concurrent execution of two or more parts of a program (threads) for maximum utilization of CPU. A thread is a lightweight sub-process and the smallest unit of processing.

Program for Inter-thread Communication:

```
class SharedResource { boolean flag = false; synchronized void produce() throws InterruptedException {
if(flag) wait(); System.out.println("Producing..."); flag = true; notify(); } synchronized void consume() throws
InterruptedException { if(!flag) wait(); System.out.println("Consuming..."); flag = false; notify(); } } class
Producer extends Thread { SharedResource r; Producer(SharedResource r) { this.r = r; } public void run() { try
{ for(int i=0; i<5; i++) r.produce(); } catch(InterruptedException e) { e.printStackTrace(); } } } class Consumer
extends Thread { SharedResource r; Consumer(SharedResource r) { this.r = r; } public void run() { try { for(int
i=0; i<5; i++) r.consume(); } catch(InterruptedException e) { e.printStackTrace(); } } } public class
InterThreadDemo { public static void main(String[] args) { SharedResource r = new SharedResource(); new
Producer(r).start(); new Consumer(r).start(); } }
```

Explanation: wait() makes the thread wait until notified. notify() wakes up a waiting thread. This achieves inter-thread communication (producer-consumer pattern).

b) Stream:

A Stream in Java is a sequence of data (bytes or characters) that flows from a source to a destination. Java I/O streams are used for reading from and writing to files, network sockets, and other I/O devices.

Program to copy content from one file to another:

```
import java.io.*; public class FileCopy { public static void main(String[] args) { try { FileInputStream fis = new
FileInputStream("source.txt"); FileOutputStream fos = new FileOutputStream("destination.txt"); int ch;
while((ch = fis.read()) != -1) { fos.write(ch); } System.out.println("File copied successfully!"); fis.close();
fos.close(); } catch(IOException e) { System.out.println("Error: " + e.getMessage()); } }
```

Explanation: FileInputStream reads bytes from the source file. FileOutputStream writes bytes to the destination file. The while loop reads one byte at a time until EOF (-1) is reached.

-
10. a) Define Collection Class. Explain different Wrapper classes in java. [1 + 4]
b) Define AWT. Explain different types of Layout Managers in java. [1 + 4]

[10]

Answer: a) Collection Class:

A Collection in Java is a framework that provides an architecture to store and manipulate a group of objects. It includes interfaces (List, Set, Queue, Map) and classes (ArrayList, LinkedList, HashSet, TreeSet, HashMap, etc.). Collections provide methods for searching, sorting, insertion, manipulation, and deletion of data.

Wrapper Classes in Java:

Wrapper classes convert primitive data types into objects. Each primitive type has a corresponding wrapper class:

PrimitiveWrapper Class byteByte shortShort intInteger longLong floatFloat doubleDouble charCharacter
booleanBoolean

Need for Wrapper Classes:

- Collections (ArrayList, HashSet) can store only objects, not primitives.
- Wrapper classes provide useful methods (parseInt(), toString(), compareTo()).
- Autoboxing (primitive → object) and Unboxing (object → primitive) are automatic in Java.

b) AWT (Abstract Window Toolkit):

AWT is Java's original platform-dependent GUI (Graphical User Interface) toolkit. It contains classes for creating windows, buttons, text fields, menus, and other UI components. AWT components are heavyweight because they use native OS resources.

Layout Managers in Java AWT:

1. **FlowLayout:** Arranges components in a left-to-right, top-to-bottom flow (like lines of text in a paragraph). Default for Panel.
2. **BorderLayout:** Divides the container into five regions: NORTH, SOUTH, EAST, WEST, and CENTER. Default for Frame and Dialog.
3. **GridLayout:** Arranges components in a rectangular grid (rows × columns). All cells have equal size.
4. **CardLayout:** Treats each component as a card. Only one card is visible at a time, useful for wizards or tabbed interfaces.
5. **GridBagLayout:** Most flexible layout manager. Components can span multiple rows/columns and have different sizes. Uses GridBagConstraints for positioning.

11. a) List and explain any five swing controls with their uses. [5]
b) Define JDBC. Write a program to display all records from a table of database. [1 + 4]

[10]

Answer: a) Five Swing Controls with Uses:

1. JButton: A push button that triggers an action when clicked. Used for submitting forms, confirming actions, etc.

```
JButton btn = new JButton("Click Me");
```

2. JTextField: A single-line text input field. Used for entering short text like names, passwords, search queries.

```
JTextField tf = new JTextField(20);
```

3. JComboBox: A dropdown list that allows selecting one item from a list. Used for selecting country, gender, department, etc.

```
String[] items = {"Nepal", "India", "China"}; JComboBox cb = new JComboBox(items);
```

4. JTable: Displays data in rows and columns (tabular format). Used for showing database records, spreadsheets, etc.

```
String[][] data = {{ "1", "Ram" }, { "2", "Sita" }}; String[] cols = { "ID", "Name" }; JTable table = new JTable(data, cols);
```

5. JRadioButton: A selectable button where only one option can be selected from a group. Used for single-choice questions, gender selection, etc.

```
JRadioButton male = new JRadioButton("Male"); JRadioButton female = new JRadioButton("Female");  
ButtonGroup bg = new ButtonGroup(); bg.add(male); bg.add(female);
```

b) JDBC (Java Database Connectivity):

JDBC is a Java API that enables Java applications to interact with relational databases. It provides methods to connect to a database, execute SQL queries, and retrieve results.

Steps to use JDBC:

1. Load the JDBC driver: **Class.forName("com.mysql.cj.jdbc.Driver");**
2. Establish connection: **DriverManager.getConnection(url, user, password);**
3. Create Statement: **stmt = conn.createStatement();**
4. Execute query: **ResultSet rs = stmt.executeQuery("SELECT * FROM table");**
5. Process results and close resources.

Program to display all records:

```
import java.sql.*; public class JDBCdemo { public static void main(String[] args) { try {  
Class.forName("com.mysql.cj.jdbc.Driver"); Connection conn = DriverManager.getConnection(  
"jdbc:mysql://localhost:3306/college", "root", "password"); Statement stmt = conn.createStatement();  
ResultSet rs = stmt.executeQuery("SELECT * FROM students"); System.out.println("ID Name Age");  
while(rs.next()) { System.out.println(rs.getInt("id") + " " + rs.getString("name") + " " + rs.getInt("age")); }  
rs.close(); stmt.close(); conn.close(); } catch(Exception e) { e.printStackTrace(); } } }
```

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.